Arduino-Based Infrared RC Car

Everton Smith evertonwsmith@gmail.com Evertonwsmith.github.io/Everton

Contents

Project Overview 1
Hardware Components 2
System Architecture & Wiring 2
Motor Control Pinout 3
Peripheral & Input Pinout 3
Schematic and Wiring Diagrams 4
Software Implementation 5
State Machine Logic 5
PWM Speed Control 6
Safety Features 6
Code Base 6

Project Overview

This personal project was aimed at creating a 4 motor Infrared Receiver Remote Controlled Car. The system uses an **Arduino Uno** to control components, 2 L293D Quadruple Half-H Drivers and an IR Receiver for wireless input. The RC Car features speed control, basic directional controls, and peripheral lighting.

Hardware Components

The system is built using the following core components as seen in the provided schematics:

- **Microcontroller:** Arduino Uno R3.
- **Motor Driver:** L293D Integrated Circuit (IC).
- **Actuators:** 4x DC Motors (Front Left, Front Right, Back Left, Back Right).
- **Input:** IR Receiver (TSOP or similar).
- **Power Source:** 6V Battery Pack (4x 1.5V AA Batteries).
- **Peripherals:** Orange LEDs (Headlights), Red LEDs (Taillights)

System Architecture & Wiring

The vehicle's logic is distributed across digital and analog pins to manage high-current motors and signal processing simultaneously.

Motor Control Pinout

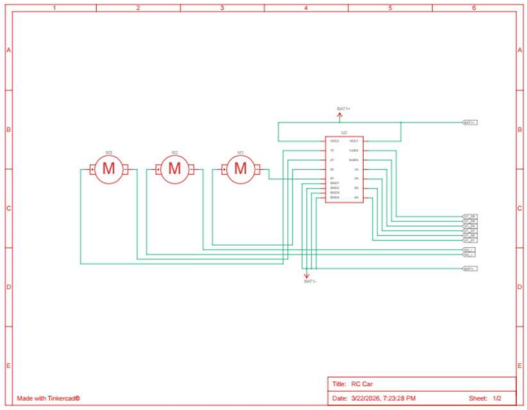
The motors are controlled via the L293D driver to allow for bi-directional movement and PWM-based speed control.

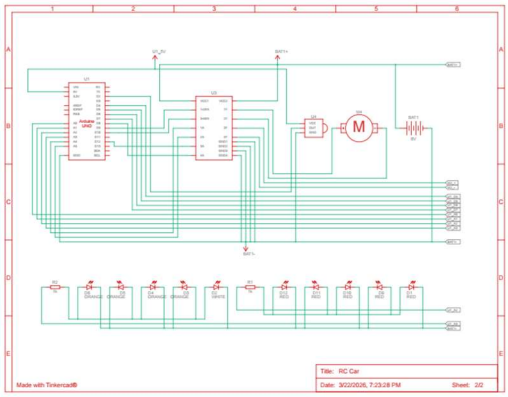
Motor Position	Power (PWM)	Input 1	Input 2
Front Left	Pin 5	Pin 15 (A1)	Pin 14 (A0)
Back Left	Pin 6	Pin 4	Pin 7
Front Right	Pin 10	Pin 8	Pin 12
Back Right	Pin 9	Pin 18 (A4)	Pin 19 (A5)

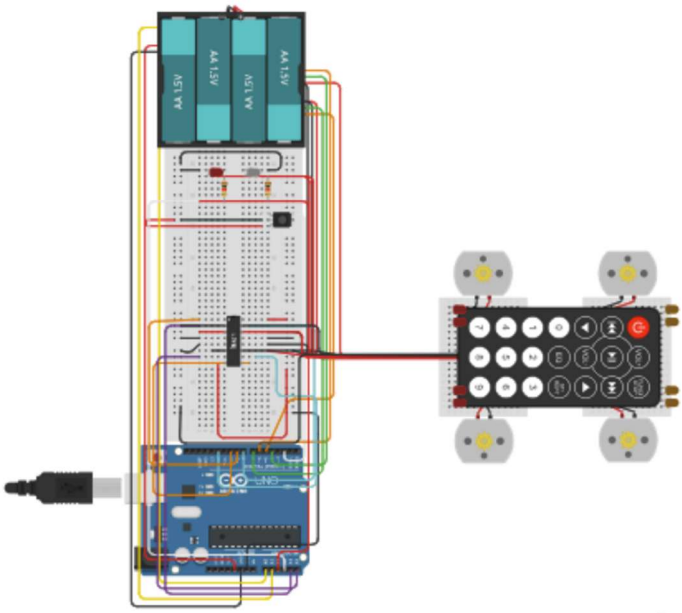
Peripheral & Input Pinout

- **IR Receiver:** Digital Pin 2.
- **Headlights:** Pin 17 (A3).
- **Taillights:** Pin 16 (A2).

Schematic and Wiring Diagrams







Software Implementation

The firmware is written in C++ using the Arduino framework and the IRremote.h library for signal decoding.

State Machine Logic

The code utilizes an **Enumerated Type (RobotState)** to manage the vehicle's behavior. This prevents conflicting commands and ensures smooth transitions between movements.

State	**Action**	**IR Command Code**
STATE_IDLE	All motors stop; Taillights ON.	5
STATE_FORWARD	All motors move forward; Headlights ON.	1
STATE_BACKWARD	All motors reverse; Taillights ON.	9
STATE_LEFT	Left wheels reverse, Right wheels forward (Pivot).	6
STATE_RIGHT	Right wheels reverse, Left wheels forward (Pivot).	4
STATE_SPEED_UP	Increments MOTOR_SPEED by 10 (Max 255).	10
STATE_SLOW_DOWN	Decrements MOTOR_SPEED by 10 (Min 0).	8
STATE_EMERGENCY	Flashing lights; system lockout until reset.	12

PWM Speed Control

The car initializes at a default speed of **150** (on a 0-255 scale). The MOTOR_SPEED_UP() and MOTOR_SPEED_DOWN() functions dynamically update the analogWrite values across all four power pins, allowing the user to adjust velocity in real-time.

Safety Features

- **Emergency Mode:** Triggered by IR command 12, this state halts all motor functions and enters a while loop that flashes the headlights and taillights every 1000ms until a new valid command is received.
- **Visual Feedback:** The taillights automatically engage during STOP and BACKWARD states to simulate realistic vehicle braking and reversing signals.

Code Base

```
#include <IRremote.h>

//*************************************************************************
//Formatting Constants
const String line = "*****************************************************";
//*************************************************************************



//*************************************************************************
// Initiate state
// _________   _________
// |FLIN1:15|   |FRIN1:8 |
// |FLIN2:14|   |FRIN2:12|
//       \     /
//        \   /
//   ______\_/_______
```

```
//  | FLPWR - 5      |
//  | BLPWR - 6      |
//  | FRPWR - 10     |
//  | BRPWR - 9      |
//  \---------------/
//        /    \
//       /      \
//      /        \
// |BLIN1:4|    |BRIN1:18|
// |BLIN2:7|    |BRIN2:19|
//*************************************************************************

// Front Left (Timer 0)
const int FRONT_LEFT_ANALOG_PWR = 5;
const int FRONT_LEFT_INPUT_1 = 15;  // A1
const int FRONT_LEFT_INPUT_2 = 14;  // A0

// Back Left (Timer 0)
const int BACK_LEFT_ANALOG_PWR = 6;
const int BACK_LEFT_INPUT_1 = 4;
const int BACK_LEFT_INPUT_2 = 7;

// Front Right (Timer 1)
const int FRONT_RIGHT_ANALOG_PWR = 10;
const int FRONT_RIGHT_INPUT_1 = 8;
const int FRONT_RIGHT_INPUT_2 = 12;

// Back Right (Timer 1)
const int BACK_RIGHT_ANALOG_PWR = 9;
const int BACK_RIGHT_INPUT_1 = 18;  // A4
const int BACK_RIGHT_INPUT_2 = 19;  // A5

// Peripherals
const int HEADLIGHTS = 17;  // A3
const int TAILLIGHTS = 16;  // A2
const int IR_INPUT_PIN =2;


//*************************************************************************
// Initiate states
// ____________________
//| Code   |  Meaning    |
//|  0     |  IDLE       |
//|  1     |  FORWARDS   |
//|  2     |  BACKWARDS  |
```

```
//|  3     |  LEFT       |
//|  4     |  RIGHT      |
//|  5     |  SPEED-UP   |
//|  6     |  SLOW-DOWN  |
//|  7     |  EMERGENCY  |
//------------------------
//*****************************************************************************
enum RobotState {
  STATE_IDLE,
  STATE_FORWARD,
  STATE_BACKWARD,
  STATE_LEFT,
  STATE_RIGHT,
  STATE_SPEED_UP,
  STATE_SLOW_DOWN,
  STATE_EMERGENCY
};

RobotState currentState = STATE_IDLE;
RobotState previousState = STATE_IDLE;
int MOTOR_SPEED = 150;

void setup() {
  Serial.begin(9600);
  //*****************************************************************************
  // Initialize all pins as OUTPUT
  //*****************************************************************************
  int OUTPUT_PINS[] = { FRONT_LEFT_ANALOG_PWR, FRONT_RIGHT_ANALOG_PWR,
BACK_LEFT_ANALOG_PWR, BACK_RIGHT_ANALOG_PWR,
                        FRONT_LEFT_INPUT_1, FRONT_LEFT_INPUT_2,
BACK_LEFT_INPUT_1, BACK_LEFT_INPUT_2,
                        FRONT_RIGHT_INPUT_1, FRONT_RIGHT_INPUT_2,
BACK_RIGHT_INPUT_1, BACK_RIGHT_INPUT_2,
                        HEADLIGHTS, TAILLIGHTS };

  SET_PINS(OUTPUT_PINS);
  // PRINT_SERIAL("OUTPUT PINS ASSIGNED");
  //*****************************************************************************
  // Start IR Receiver (Uses Timer 2 internally)
  //*****************************************************************************
  IrReceiver.begin(IR_INPUT_PIN, true);
  // PRINT_SERIAL("SYSTEM READY - IR: TIMER 2");
  digitalWrite(HEADLIGHTS, HIGH);
}
```

```
//*****************************************************************************
//Begin loop
//*****************************************************************************
void loop() {
  // 1. Process Input (Always check for IR or sensors)
  checkInputs();
  // 2. Perform logic based on state
  switch (currentState) {
    case STATE_IDLE:
      STOP();
      break;
    case STATE_FORWARD:
      FORWARD();
      break;
    case STATE_BACKWARD:
      BACKWARD();
      break;
    case STATE_LEFT:
      TURN_LEFT();
      break;
    case STATE_RIGHT:
      TURN_RIGHT();
      break;
    case STATE_SPEED_UP:
      MOTOR_SPEED_UP();
      currentState = previousState;
      break;
    case STATE_SLOW_DOWN:
      MOTOR_SPEED_DOWN();
      currentState = previousState;
      break;
    case STATE_EMERGENCY:
      EMERGENCY();
      break;
  }
  RobotState previousState = currentState;
}
void checkInputs() {
  if (IrReceiver.decode()) {
    int command = IrReceiver.decodedIRData.command;
    // PRINT_SERIAL_COMMAND(command);

    // Map IR commands to States
    switch (command) {
      case 1: currentState = STATE_FORWARD; break;
```

```
      case 9: currentState = STATE_BACKWARD; break;
      case 5: currentState = STATE_IDLE; break;
      case 6: currentState = STATE_LEFT; break;
      case 4: currentState = STATE_RIGHT; break;
      case 8: currentState = STATE_SLOW_DOWN; break;
      case 10: currentState = STATE_SPEED_UP; break;
      case 12: currentState = STATE_EMERGENCY; break;
    }

    IrReceiver.resume();  // Ready for next signal
  }
}
void FORWARD() {
  // PRINT_SERIAL("FORWARD");
  digitalWrite(HEADLIGHTS, HIGH);
  digitalWrite(TAILLIGHTS, LOW);
  previousState = STATE_FORWARD;
  digitalWrite(TAILLIGHTS, LOW);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  digitalWrite(FRONT_LEFT_INPUT_1, LOW);
  digitalWrite(FRONT_LEFT_INPUT_2, HIGH);
  digitalWrite(BACK_LEFT_INPUT_1, HIGH);
  digitalWrite(BACK_LEFT_INPUT_2, LOW);
  digitalWrite(FRONT_RIGHT_INPUT_1, LOW);
  digitalWrite(FRONT_RIGHT_INPUT_2, HIGH);
  digitalWrite(BACK_RIGHT_INPUT_1, HIGH);
  digitalWrite(BACK_RIGHT_INPUT_2, LOW);
}
void BACKWARD() {
  // PRINT_SERIAL("BACKWARDS");
  previousState = STATE_BACKWARD;
  digitalWrite(TAILLIGHTS, HIGH);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  digitalWrite(FRONT_LEFT_INPUT_1, HIGH);
  digitalWrite(FRONT_LEFT_INPUT_2, LOW);
  digitalWrite(BACK_LEFT_INPUT_1, LOW);
  digitalWrite(BACK_LEFT_INPUT_2, HIGH);
  digitalWrite(FRONT_RIGHT_INPUT_1, HIGH);
  digitalWrite(FRONT_RIGHT_INPUT_2, LOW);
```

```
  digitalWrite(BACK_RIGHT_INPUT_1, LOW);
  digitalWrite(BACK_RIGHT_INPUT_2, HIGH);
}
void TURN_LEFT() {
  // PRINT_SERIAL("LEFT");
  digitalWrite(HEADLIGHTS, HIGH);
  digitalWrite(TAILLIGHTS, LOW);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  digitalWrite(FRONT_LEFT_INPUT_1, LOW);
  digitalWrite(FRONT_LEFT_INPUT_2, HIGH);
  digitalWrite(BACK_RIGHT_INPUT_1, LOW);
  digitalWrite(BACK_RIGHT_INPUT_2, HIGH);
  digitalWrite(FRONT_RIGHT_INPUT_1, HIGH);
  digitalWrite(FRONT_RIGHT_INPUT_2, LOW);
  digitalWrite(BACK_LEFT_INPUT_1, HIGH);
  digitalWrite(BACK_LEFT_INPUT_2, LOW);
}
void TURN_RIGHT() {
  // PRINT_SERIAL("TURN_RIGHT");
  digitalWrite(HEADLIGHTS, HIGH);
  digitalWrite(TAILLIGHTS, LOW);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  digitalWrite(FRONT_LEFT_INPUT_1, HIGH);
  digitalWrite(FRONT_LEFT_INPUT_2, LOW);
  digitalWrite(BACK_RIGHT_INPUT_1, HIGH);
  digitalWrite(BACK_RIGHT_INPUT_2, LOW);
  digitalWrite(FRONT_RIGHT_INPUT_1, LOW);
  digitalWrite(FRONT_RIGHT_INPUT_2, HIGH);
  digitalWrite(BACK_LEFT_INPUT_1, LOW);
  digitalWrite(BACK_LEFT_INPUT_2, HIGH);
}
void STOP() {
  // PRINT_SERIAL("STOP");
  digitalWrite(TAILLIGHTS, HIGH);
  digitalWrite(FRONT_LEFT_INPUT_1, LOW);
  digitalWrite(FRONT_LEFT_INPUT_2, LOW);
  digitalWrite(BACK_LEFT_INPUT_1, LOW);
  digitalWrite(BACK_LEFT_INPUT_2, LOW);
  digitalWrite(FRONT_RIGHT_INPUT_1, LOW);
```

```
  digitalWrite(FRONT_RIGHT_INPUT_2, LOW);
  digitalWrite(BACK_RIGHT_INPUT_1, LOW);
  digitalWrite(BACK_RIGHT_INPUT_2, LOW);
}
void EMERGENCY() {
  // PRINT_SERIAL("ERROR ERROR ERROR");
  STOP();
  while (currentState == STATE_EMERGENCY) {
    digitalWrite(HEADLIGHTS, HIGH);
    digitalWrite(TAILLIGHTS, HIGH);
    delay(500);
    digitalWrite(HEADLIGHTS, LOW);
    digitalWrite(TAILLIGHTS, LOW);
    delay(500);
    checkInputs();
  }
}
void MOTOR_SPEED_UP() {
  // PRINT_SERIAL("MOTOR_SPEED_UP");
  MOTOR_SPEED += 10;
  if (MOTOR_SPEED > 255) MOTOR_SPEED = 255;
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
}
void MOTOR_SPEED_DOWN() {
  // PRINT_SERIAL("MOTOR_SPEED_DOWN");
  MOTOR_SPEED -= 10;
  if (MOTOR_SPEED < 0) MOTOR_SPEED = 0;
  analogWrite(BACK_LEFT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(BACK_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_RIGHT_ANALOG_PWR, MOTOR_SPEED);
  analogWrite(FRONT_LEFT_ANALOG_PWR, MOTOR_SPEED);
}
void PRINT_SERIAL(String s) {
  Serial.print(line);
  Serial.print("\n");
  Serial.print(s);
  Serial.print("\n");
  Serial.print(line);
  Serial.print("\n\n");
}
void PRINT_SERIAL_COMMAND(int n) {
  Serial.print(line);
```

```
  Serial.print("\n");
  Serial.print("Command: ");
  Serial.print(n);
  Serial.print("\n");
  Serial.print(line);
  Serial.print("\n\n");
}
void SET_PINS(int a[]) {
  for (int i = 0; i < 14; i++) {
    pinMode(a[i], OUTPUT);
  }
}
```